



Xemtailieu ngân hàng câu hỏi môn hệ điều hành kèm đáp án

hệ điều hành (Học viện Công nghệ Bưu chính Viễn thông)



Scan to open on Studeersnel

1. Ngân hàng câu hỏi thi

• Câu hỏi loại 1 điểm

Chương 1 :

Câu hỏi 1.1: Chương trình ứng dụng gọi dịch vụ của hệ điều hành bằng cách nào? Hãy lấy một ví dụ về giao diện lập trình cho một hệ điều hành thông dụng.

Để các chương trình có thể sử dụng được những dịch vụ HDH cung cấp giao diện lập trình

Vd :

Câu hỏi 1.2: Trình bày kỹ thuật xử lý theo mẻ (lô) và ưu điểm của kỹ thuật này. Hệ thống xử lý theo mẻ có cần hệ điều hành không ?

Xử lý theo mẻ:

☞ Chương trình được phân thành các mẻ: gồm những chương trình có yêu cầu giống nhau

☞ Toàn bộ mẻ được nạp vào băng từ và được tải vào máy để thực hiện lần lượt

☞ Chương trình giám sát (monitor): tự động nạp chương trình tiếp theo vào máy và cho phép nó chạy

☞ => Giảm đáng kể thời gian chuyển đổi giữa hai chương trình trong cùng một mẻ

☞ Trình giám sát là dạng đơn giản nhất của HDH

_ Khi hệ thống xử lý theo mẻ ko cần HDH

Câu hỏi 1.3: Đa chương trình là gì ? Lý do sử dụng đa chương trình trong máy tính ? Yêu cầu đối với phần cứng khi sử dụng đa chương trình?

- _ Hệ thống chứa đồng thời nhiều chương trình trong bộ nhớ
- _ Khi một chương trình phải dừng lại để thực hiện vào ra, HDH sẽ chuyển CPU sang thực hiện một chương trình khác
- => Giảm thời gian chạy không tải của CPU
- Thời gian chờ đợi của CPU trong chế độ đa chương trình giảm đáng kể so với trong trường hợp đơn chương trình
- ☞ HDH phức tạp hơn rất nhiều so với HDH đơn chương trình
- _ Đòi hỏi hỗ trợ từ phần cứng, đặc biệt khả năng vào/ra bằng ngắt và DMA

Chương 2 :

Câu hỏi 1.4 : Trình bày khái niệm tiến trình và chỉ rõ điểm khác nhau giữa tiến trình với chương trình. Nêu tên ít nhất bốn thao tác liên quan tới quản lý tiến trình (chỉ cần nêu tên, không cần trình bày chi tiết).

- ☞ Tiến trình là một chương trình đang trong quá trình thực hiện

Chương trình	Tiến trình
Thực thể tĩnh Thực thể động Không sở hữu tài nguyên cụ thể	Được cấp một số tài nguyên để chứa tiến trình và thực hiện lệnh

- _ Thao tác liên quan tới quản lý tiến trình : tạo mới tiến trình, kết thúc tiến trình, chuyển đổi giữa các tiến trình

Câu hỏi 1.5 : Trình bày về thao tác tạo mới tiến trình. Tiến trình có thể bị kết thúc trong những trường hợp nào ?

Gán số định danh cho tiến trình được tạo mới và tạo một ô trong bảng tiến trình

- ☞ Tạo không gian nhớ cho tiến trình và PCB ☞ Khởi tạo PCB

📖 Liên kết PCB của tiến trình vào các danh sách quản lý

Tiến trình có thể bị kết thúc trong 2TH: kết thúc bình thường và bị kết thúc

Câu hỏi 1.6 : Trình bày về thao tác và quá trình chuyển đổi giữa các tiến trình.

Thông tin về tiến trình hiện thời (chứa trong PCB) được gọi là ngữ cảnh (context) của tiến trình

Việc chuyển giữa tiến trình còn được gọi là chuyển đổi ngữ cảnh

Xảy ra khi:

📖 Có ngắt

📖 Tiến trình gọi lời gọi hệ thống

Trước khi chuyển sang thực hiện tiến trình khác, ngữ cảnh được lưu vào PCB

Khi được cấp phát CPU thực hiện trở lại, ngữ cảnh được khôi phục từ PCB vào các thanh ghi và bảng tương ứng
Sau khi thực hiện ngắt, hệ thống thực hiện tiến trình khác

📖 Thay đổi trạng thái tiến trình

📖 Cập nhật thông tin thống kê trong PCB

📖 Chuyển liên kết PCB của tiến trình vào danh sách ứng với trạng thái mới

📖 Cập nhật PCB của tiến trình mới được chọn

📖 Cập nhật nội dung thanh ghi và trạng thái CPU

📖 => Chuyển đổi tiến trình đòi hỏi thời gian

Tiến trình được xem xét từ 2 khía cạnh:

📖 Tiến trình là 1 đơn vị sở hữu tài nguyên

📖 Tiến trình là 1 đơn vị thực hiện công việc tính toán xử lý

Các HĐH trước đây: mỗi tiến trình chỉ tương ứng với 1 đơn vị xử lý duy nhất

=> Tiến trình không thể thực hiện nhiều hơn một công việc cùng một lúc

**Câu hỏi 1.7: Thế nào là dòng (thread) mức người dùng và mức nhân.
Nêu ưu nhược điểm của mỗi loại.**

So Sánh	Mức ng dùng	Mức Nhân
	Do trình ứng dụng tự tạo ra và quản lý Sử dụng thư viện do ngôn ngữ lập trình cung cấp HDH vẫn coi tiến trình như một đơn vị duy nhất với một trạng thái duy nhất Việc phân phối CPU được thực hiện cho cả tiến trình	HDH cung cấp giao diện lập trình: gồm các lời gọi hệ thống mà trình ứng dụng có thể yêu cầu tạo/ xóa luồng Tăng tính đáp ứng và khả năng thực hiện đồng thời của các luồng trong cùng tiến trình Tạo và chuyển đổi luồng thực hiện trong chế độ nhân => tốc độ chậm
Ưu điểm	☞ Việc chuyển đổi luồng không đòi hỏi chuyển sang chế độ nhân => tiết kiệm thời gian ☞ Trình ứng dụng có thể điều độ theo đặc điểm riêng của mình, không phụ thuộc vào cách điều độ của HDH ☞ Có thể sử dụng trên cả những HDH không	

	hỗ trợ đa luồng	
Nhược điểm	Khi một luồng gọi lời gọi hệ thống và bị phong tỏa, toàn bộ tiến trình bị phong tỏa => không cho phép tận dụng ưu điểm về tính đáp ứng của mô hình đa luồng Không cho phép tận dụng kiến trúc nhiều CPU	

Câu hỏi 1.8 : Trình bày về điều độ quay vòng. Cho ví dụ minh họa về tính thời gian chờ đợi trung bình khi điều độ theo kiểu này.

Hết lượng tử thời gian mà tiến trình chưa kết thúc:

- 📖 Đồng hồ sinh ngắt
- 📖 Tiến trình đang thực hiện bị dừng lại
- 📖 Quyền điều khiển chuyển cho hàm xử lý ngắt của HDH
- 📖 HDH chuyển tiến trình về cuối hàng đợi, lấy tiến trình ở đầu và tiếp tục
- 📖 Cải thiện thời gian đáp ứng so với FCFS
- 📖 Thời gian chờ đợi trung bình vẫn dài
- 📖 Lựa chọn độ dài lượng tử thời gian?

VD ;

Câu hỏi 1.9 : Thế nào là bế tắc ? Điều kiện xảy ra bế tắc là gì ?

Tình trạng một nhóm tiến trình có cạnh tranh về tài nguyên

hay có hợp tác phải dừng vô hạn

Do tiến trình phải chờ đợi một sự kiện chỉ có thể sinh ra

bởi tiến trình khác cũng đang trong trạng thái chờ đợi

Đồng thời xảy ra 4 điều kiện:

1. Loại trừ tương hỗ: có tài nguyên nguy hiểm, tại 1 thời điểm duy nhất 1

2. tiến

trình sử dụng

2. Giữ và chờ: tiến trình giữ tài nguyên trong khi chờ đợi

3. Không có phân phối lại (no preemption): tài nguyên do tiến trình giữ không thể phân phối lại cho tiến trình khác trừ khi tiến trình đang giữ tự nguyện

giải phóng tài nguyên

3. Chờ đợi vòng tròn: tồn tại nhóm tiến trình $P_1, P_2, \dots, P_n$ sao cho P_1

4. chờ

đợi tài nguyên do P_2 đang giữ, P_2 chờ tài nguyên do P_3 đang giữ, ..., P_n

chờ tài

nguyên do P_1 đang giữ

Chương 3 :Câu hỏi 1.10 : Thế nào là địa chỉ lô gic và địa chỉ vật lý ?

☞ Địa chỉ logic:

☞ Gán cho các lệnh và dữ liệu không phụ thuộc vào vị trí cụ thể tiến trình trong bộ nhớ

☞ Chương trình ứng dụng chỉ nhìn thấy và làm việc với địa chỉ logic này

☞ Là địa chỉ tương đối tức là mỗi phần tử của chương

trình được gán một địa chỉ tương đối đối với một vị trí nào đó

- ☞ Địa chỉ vật lý:

- ☞ Là địa chỉ chính xác trong bộ nhớ máy tính

- ☞ Các mạch nhớ sử dụng để truy nhập tới chương trình và dữ liệu

Câu hỏi 1.11 : Trình bày kỹ thuật phân chương cố định bộ nhớ.(SGK 15)

Chia MEM thành các chương với số lượng nhất định, không thay đổi, gán cho tiến trình 1 chương chứa data, lệnh

Kích thước các chương bằng nhau:

- ☞ Đơn giản

- ☞ Kích thước chương trình > kích thước chương => không thể cấp phát

- ☞ Gây phân mảnh trong

Kích thước các chương khác nhau:

- Chọn chương có kích thước nhỏ nhất: cần có hàng đợi lệnh cho mỗi chương:

- ☞ Giảm phân mảnh trong, tối ưu cho từng chương

- ☞ Hệ thống không tối ưu

- Kích thước các chương khác nhau:

- ☞ Dùng hàng đợi chung cho mọi chương:

- ☞ Chương sẵn có nhỏ nhất sẽ được cấp phát

- ☞ Khi 1 chương được giải

phóng: chọn tiến trình gần

đầu hàng độ nhất và có

kích thước phù hợp nhất

- ☞ Ưu điểm: đơn giản, ít xử lý

☞ Nhược điểm:

☞ Số lượng chương xác định tại thời điểm tạo hệ thống hạn chế số lượng tiến trình hoạt động

☞ Kích thước chương thiết lập trước: không hiệu quả

Câu hỏi 1.12 : Trình bày cơ chế ánh xạ địa chỉ khi sử dụng kỹ thuật phân chương bộ nhớ.(24)

Vị trí các chương thường không biết trước và có thể thay đổi

=> cần có cơ chế biến đổi địa chỉ logic thành vật lý

Cấm truy cập trái phép: tiến trình này truy cập tới phần MEM của tiến trình khác

Ánh xạ địa chỉ do phần cứng đảm nhiệm

Khi tiến trình được tải vào MEM, CPU dành 2 thanh ghi:

☞ Thanh ghi cơ sở: chứa địa chỉ bắt đầu của tiến trình

☞ Thanh ghi giới hạn: chứa độ dài chương

Địa chỉ logic được so sánh với nội dung của thanh ghi giới hạn

☞ Nếu lớn hơn: lỗi truy cập

☞ Nhỏ hơn: được đưa tới bộ cộng với thanh ghi cơ sở để thành địa chỉ vật lý

Nếu chương bị di chuyển thì nội dung của thanh ghi cơ sở bị thay đổi chứa địa chỉ vị trí mới

Câu hỏi 1.13 : Trình bày phương pháp kết hợp phân trang với phân đoạn. Vẽ sơ đồ và giải thích cơ chế ánh xạ địa chỉ.

Phân đoạn chương trình, mỗi đoạn sẽ tiến hành phân trang

Địa chỉ gồm: số thứ tự đoạn, số thự tự trang, độ dịch trong trang

Tiến trình có 1 bảng phân đoạn, mỗi đoạn có 1 bảng phân trang

Hình sgk 44

Chương 4

Câu hỏi 1.14 : Việc định nghĩa và sử dụng khái niệm file đem lại những ưu điểm gì ? Khi đặt tên cho file cần quan tâm tới những quy định gì ?

Việc định nghĩa và sử dụng khái niệm file đem cho việc sd file 1 cách dễ dàng.

Đặt tên cho file:

- Cho phép xd File
- Là thông tin ng dùng thường sd khi làm việc với file
- Quy tắc đặt tên cho file phụ thuộc vào HDH:
 - + Độ dài tối đa : vd là 255 ky tu cho ca ten file va đường dẫn(Windows NT FAT)
 - + Phân biệt chữ hoa chữ thường: vd: Windows NT FAT ko phân biệt
 - + Cho phép sd dấu cách: vd: Windows NT FAT là có
 - + Các kí tự cấm: vd : Windows NT FAT là bắt đầu bằng chữ cái hoặc số, ko dc chứa các kí tự / \ [] : ; | = , ^ ? @

Câu hỏi 1.15 : Trình bày khái niệm thư mục ? Thông tin trong các khoản mục có nhất thiết phải lưu trữ gần nhau không ?

+Thư mục = $\sum$ các khoản mục ~ files

Khoản mục chứa các thông tin về file: tên, kích thước, vị trí, kiểu file,...hoặc con trỏ tới nơi lưu trữ thông tin này.

Coi thư mục như 1 bảng, mỗi dòng là khoản mục ứng với 1 file.

+ Thông tin trong các khoản mục phải nhất thiết lưu trữ gần nhau vì

- Toàn bộ thuộc tính của File dc lưu trong thư mục, file chỉ chứa data=> kích thước khoản mục, thư mục lớn.

- Thư mục chỉ lưu thông tin tối thiểu cần thiết cho việc tìm kiếm vị trí file trên đĩa=> kích thước giảm.

• **Câu hỏi loại 2 điểm**

Chương 1 :

Câu hỏi 2.1: Trình bày ngắn gọn về các thành phần cơ bản của hệ điều hành.

- Quản lý tiến trình
- Quản lý bộ nhớ
- Quản lý vào ra
- Quản lý tệp và thư mục
- Hỗ trợ mạng và quản lý phân tán
- Giao diện với người dùng
- Các chương trình tiện ích và ứng dụng

Câu hỏi 2.2 : Trình bày về nhân của hệ điều hành ? Thế nào là chế độ nhân và chế độ người dùng ?

Nhân (kernel) là phần cốt lõi, thực hiện các chức năng cơ bản nhất, quan trọng nhất của HDH và thường xuyên được giữ trong bộ nhớ

HDH gồm nhiều thành phần, chỉ tải những thành phần quan trọng không thể thiếu được vào bộ nhớ gọi là nhân

Nhân chạy trong chế độ đặc quyền – chế độ nhân

Các chương trình bình thường chạy trong chế độ người dùng

Câu hỏi 2.3 : Trình bày về cấu trúc nguyên khối và cấu trúc phân lớp của hệ điều hành. Phân tích so sánh ưu nhược điểm hai kiểu cấu trúc này.

So sánh	Nguyên khối	Phân lớp
	Toàn bộ chương trình và dữ liệu của HDH có chung 1 không gian nhớ  HDH trở thành một tập hợp các thủ tục hay các chương trình con	Các thành phần được chia thành các lớp nằm chồng lên nhau Mỗi lớp chỉ có thể liên lạc với lớp nằm kề bên trên và kề bên dưới Mỗi lớp chỉ có thể sử dụng dịch vụ do lớp nằm ngay bên dưới

		cung cấp
Ưu điểm	nhANH	dễ xây dựng, dễ sửa lỗi
Nhược điểm	không an toàn, không mềm dẻo	tốc độ chậm hơn cấu trúc nguyên khối

Câu hỏi 2.4: Trình bày về cấu trúc vi nhân của hệ điều hành. Phân tích so sánh cấu trúc này với cấu trúc nguyên khối và cấu trúc phân lớp.

So sánh	Cấu trúc vi nhân	Nguyên khối	Phân lớp
	Nhân chỉ chứa các chức năng quan trọng nhất Các chức năng còn lại được đặt vào các modul riêng: chạy trong chế độ đặc quyền hoặc người dùng	Toàn bộ chương trình và dữ liệu của HDH có chung 1 không gian nhớ ☞ HDH trở thành một tập hợp các thủ tục hay các chương trình con	Các thành phần được chia thành các lớp nằm chồng lên nhau Mỗi lớp chỉ có thể liên lạc với lớp nằm kề bên trên và kề bên dưới Mỗi lớp chỉ có thể sử dụng dịch vụ do lớp nằm ngay bên dưới cung cấp
Ưu điểm	mềm dẻo, an toàn	nhANH	dễ xây dựng, dễ sửa lỗi
Nhược điểm	tốc độ chậm hơn so với cấu trúc nguyên khối	không an toàn, không mềm dẻo	tốc độ chậm hơn cấu trúc nguyên khối

Chương 2 :

Câu hỏi 2.5 : Trình bày về năm trạng thái của tiến trình. Vẽ sơ đồ và giải thích về việc chuyển đổi giữa năm trạng thái này

Mô hình 5 trạng thái: mới khởi tạo, sẵn sàng, chạy, chờ đợi, kết thúc

_ Mới khởi tạo: tiến trình đang được

tạo ra

Sẵn sàng: tiến trình chờ được cấp

CPU để thực hiện lệnh của mình

Chạy: lệnh của tiến trình được CP

thực hiện

Chờ đợi: tiến trình chờ đợi một sự

kiện gì đó xảy ra (blocked)

Kết thúc: tiến trình đã kết thúc việ

thực hiện nhưng vẫn chưa bị xóa

SD :SGK 5

Câu hỏi 2.6 : Điều độ tiến trình là gì ? Điều độ dòng có khác điều độ tiến trình không ? Trình bày về điều độ có phân phối lại và không phân phối lại.

📖 Điều độ (scheduling) hay lập lịch là quyết định tiến trình nào được sử dụng tài nguyên phần cứng khi nào, trong thời gian bao lâu

📖 Tập trung vào vấn đề điều độ đối với CPU

📖 => Quyết định thứ tự và thời gian sử dụng CPU

📖 **Điều độ tiến trình và điều độ dòng:**

📖 Hệ thống trước kia: tiến trình là đơn vị thực hiện chính => điều độ thực hiện với tiến trình

📖 Hệ thống hỗ trợ dòng: dòng mức nhân là đơn vị HDH cấp CPU

📖 => Sử dụng thuật ngữ điều độ tiến trình rộng rãi 🦋 điều độ dòng

📖 **Điều độ có phân phối lại (preemptive):**

📖 HDH có thể sử dụng cơ chế ngắt để thu hồi CPU của một tiến trình đang trong trạng thái chạy

📖 **Điều độ không phân phối lại (nonpreemptive):**

📖 Tiến trình đang ở trạng thái chạy sẽ được sử dụng CPU cho đến khi xảy ra một trong các tình huống sau:

📖 Tiến trình kết thúc

📖 Tiến trình phải chuyển sang trạng thái chờ đợi do thực hiện I/O

☞ => Điều độ hợp tác: chỉ thực hiện được khi tiến trình hợp tác và nhường CPU

☞ Nếu tiến trình không hợp tác, dùng CPU vô hạn => các tiến trình khác không được cấp CPU

Câu hỏi 2.7 : Trình bày hai biện pháp ngăn ngừa bế tắc (chọn 2 biện pháp bất kỳ trong số các biện pháp có thể).

☞ **Loại trừ tương hỗ: không thể ngăn ngừa**

☞ Giữ và chờ:

☞ Cách 1:

☞ Yêu cầu tiến trình phải nhận đủ toàn bộ tài nguyên cần thiết trước khi thực hiện tiếp

☞ Nếu không nhận đủ, tiến trình bị phong tỏa để chờ cho đến khi có thể nhận đủ tài nguyên

☞ Cách 2:

☞ Tiến trình chỉ được yêu cầu tài nguyên nếu không giữ tài nguyên khác

☞ Trước khi yêu cầu thêm tài nguyên, tiến trình phải giải phóng tài nguyên đã được cấp và yêu cầu lại (nếu cần) cùng với tài nguyên mới

☞ **Không có phân phối lại:**

☞ Cách 1:

☞ Khi một tiến trình yêu cầu tài nguyên nhưng không được do đã bị cấp phát, HDH sẽ thu hồi lại toàn bộ tài nguyên nó đang giữ

☞ Tiến trình chỉ có thể thực hiện tiếp sau khi lấy được tài nguyên cũ cùng với tài nguyên mới yêu cầu

☞ Cách 2:

☞ Khi tiến trình yêu cầu tài nguyên, nếu còn trống, sẽ được cấp phát ngay

☞ Nếu tài nguyên do tiến trình khác giữ mà tiến trình này đang chờ cấp thêm tài nguyên thì thu hồi lại để cấp cho tiến trình yêu cầu

☞ Nếu hai điều kiện trên đều không thỏa thì tiến trình yêu cầu tài nguyên phải chờ

Câu hỏi 2.8 : Trình bày một giải pháp giúp không xảy ra bế tắc khi sử dụng cờ hiệu cho bài toán triết gia ăn cơm.

Chương 3 :

Câu hỏi 2.9 : Trình bày kỹ thuật giúp tăng tốc độ truy cập bảng trang và bảng trang nhiều mức.

Mỗi thao tác truy cập bộ nhớ đều đòi hỏi truy cập bảng phân trang

☞ => tổ chức bảng phân trang sao cho tốc độ truy cập là cao nhất

☞ Sử dụng tập hợp các thanh ghi làm bảng phân trang:

☞ Tốc độ truy cập rất cao

☞ Số lượng thanh ghi hạn chế => không áp dụng được

☞ Giữ các bảng trang trong MEM:

☞ Vị trí mỗi bảng được trỏ bởi thanh ghi cơ sở bảng trang PTBR (Page Table Base Register)

☞ Nhiều thời gian để truy cập bảng

☞ => sử dụng bộ nhớ cache tốc độ cao

Không gian địa chỉ logic lớn (232 -> 264) => kích thước bảng trang tăng

Giả sử không gian địa chỉ logic là 232, kích thước trang là 4KB = 212

=> số lượng khoản mục cần có trong bảng trang là 220

Mỗi khoản mục có kích thước 4B

=> kích thước bảng trang là 4MB

=> cần chia bảng trang thành những phần nhỏ hơn

Tổ chức bảng trang nhiều mức: Khoản mục của bảng mức trên chỉ tới bảng trang khác

VD : SGK 39

Câu hỏi 2.10: Trình bày lý do phải đổi trang, và các bước tiến hành khi đổi trang.

Bộ nhớ ảo > bộ nhớ thực và chế độ đa chương trình -> có lúc không còn khung nào trống để nạp trang mới

Quá trình đổi trang:

☞ B1: Xác định trang cần nạp vào trên đĩa

☞ B2: Nếu có khung trống thì chuyển sang B4

☞ B3:

☞ Lựa chọn 1 khung để giải phóng, theo 1 thuật toán nào đó

☞ Ghi nội dung khung bị đổi ra đĩa (nếu cần), cập nhật bảng trang và bảng khung

☞ B4: Đọc trang cần nạp vào khung vừa giải phóng; cập nhật bảng trang và bảng khung

☞ B5: Thực hiện tiếp tiến trình từ điểm bị dừng trước khi đổi trang

Câu hỏi 2.11: Trình bày kỹ thuật đổi trang tối ưu và đổi trang vào trước ra trước.

Đổi trang tối ưu (OPT):

☞ Chọn trang sẽ không được dùng tới trong khoảng thời gian lâu nhất

để đổi

☞ Cho phép giảm tối thiểu sự kiện thiếu trang và do đó là tối ưu theo tiêu chuẩn này

☞ HDH không đoán trước được nhu cầu sử dụng các trang trong tương lai

☞ => không áp dụng trong thực tế mà chỉ để so sánh với các chiến lược khác

☞ **Vào trước ra trước (FIFO):**

☞ Trang nào được nạp vào trước thì bị đổi ra trước

☞ Đơn giản nhất

☞ Trang bị trao đổi là trang nằm lâu nhất trong bộ nhớ

Câu hỏi 2.12 : Trình bày các phương pháp xác định số lượng khung trang tối đa cấp cho mỗi tiến trình và xác định phạm vi cấp phát

- **Cấp phát slg khung cố định :**

Cấp cho tiến trình một số lượng cố định khung để chứa các trang nhớ

Số lượng được xác định vào thời điểm tạo mới tiến trình và không thay đổi trong quá trình tiến trình tồn tại

Cấp phát bằng nhau:

☞ Các tiến trình được cấp số khung tối đa bằng nhau

☞ Số lượng được xác định dựa vào kích thước MEM và mức độ đa chương trình mong muốn

Cấp phát không bằng nhau:

☞ Các tiến trình được cấp số khung tối đa khác nhau

☞ Cấp số khung tỉ lệ thuận với kích thước tiến trình

☞ Có mức ưu tiên

- **Cấp phát slg khung thay đổi :**

Số lượng khung tối đa cấp cho mỗi tiến trình có thể thay đổi trong quá trình thực hiện

Việc thay đổi phụ thuộc vào tình hình thực hiện của tiến trình

Cho phép sử dụng bộ nhớ hiệu quả hơn phương pháp cố định

=> Cần theo dõi và xử lý thông tin về tình hình sử dụng bộ nhớ của tiến trình

_ Phạm vi cấp phát:

Cấp phát toàn thể:

☞ Cho phép tiến trình đổi trang mới vào bất cứ khung nào (không bị khóa), kể cả khung đã được cấp phát cho tiến trình khác

Cấp phát cục bộ:

☞ Trang chỉ được đổi vào khung đang được cấp cho chính tiến trình đó

Phạm vi cấp phát có quan hệ mật thiết với số lượng khung tối đa:

☞ Số lượng khung cố định tương ứng với phạm vi cấp phát cục bộ

☞ Số lượng khung thay đổi tương ứng với phạm vi cấp phát toàn thể

Chương 4 :

Câu hỏi 2.13 : Trình bày các cấu trúc dữ liệu dùng cho tổ chức bên trong của thư mục.

Danh sách:

☞ Tổ chức thư mục dưới dạng danh sách các khoản mục

☞ Tìm kiếm khoản mục được thực hiện bằng cách duyệt lần lượt danh sách

☞ Thêm file mới vào thư mục:

☞ Duyệt cả thư mục để kiểm tra xem khoản mục với tên file như vậy đã có

chưa

☞ Khoản mục mới được thêm vào cuối danh sách hoặc 1 ô trong bảng

☞ Mở file, xóa file

- 📖 Tìm kiếm trong danh sách chậm

- 📖 Cache thư mục trong MEM

Cây nhị phân:

- 📖 Tăng tốc độ tìm kiếm nhờ CTDL có hỗ trợ sắp xếp

- 📖 Hệ thống file NTFS của WinNT

Bảng băm (hash table):

- 📖 Dùng hàm băm để tính vị trí của khoản mục trong thư mục theo tên file

- 📖 Thời gian tìm kiếm nhanh

- 📖 Hàm băm phụ thuộc vào kích thước của bảng băm => kích thước bảng cố định

Tổ chức thư mục của DOS:

- 📖 Mỗi đĩa logic có cây thư mục riêng, bắt đầu từ thư mục gốc ROOT

- 📖 Thư mục gốc được đặt ở phần đầu của đĩa, ngay sau sector khởi động BOOT và bảng FAT

- 📖 Thư mục gốc chứa files và các thư mục con

- 📖 Thư mục con có thể chứa files và các thư mục cấp dưới nữa

- 📖 Được tổ chức dưới dạng bảng: mỗi khoản mục chiếm 1 dòng trong bảng và có kích thước cố định 32 bytes

Tổ chức thư mục của Linux:

- 📖 Thư mục hệ thống file Ext2 của Linux có cách tổ chức đơn giản

- 📖 Khoản mục chứa tên file và địa chỉ I-node

- 📖 Thông tin còn lại về các thuộc tính file và vị trí các khối dữ liệu được lưu trên I-node chứ không phải thư mục

- 📖 Kích thước khoản mục phụ thuộc vào độ dài tên file

- 📖 Phần đầu của khoản mục có trường cho biết kích thước khoản mục

Câu hỏi 2.14 : Trình bày cách kiểm soát truy cập file sử dụng mật khẩu và sử dụng danh sách quản lý truy cập

📖 Dùng mật khẩu:

- 📖 Người dùng phải nhớ nhiều mật khẩu
- 📖 Mỗi khi thao tác với tài nguyên lại gõ mật khẩu

📖 Sử dụng danh sách quản lý truy cập ACL (Access Control List)

- 📖 Mỗi file được gán danh sách đi kèm, chứa thông tin định danh người dùng và các quyền người đó được thực hiện với file
- 📖 ACL thường được lưu trữ như thuộc tính của file/ thư mục
- 📖 Thường được sử dụng cùng với cơ chế đăng nhập
- 📖 Các quyền truy cập cơ bản:
 - 📖 Quyền đọc (r)
 - 📖 Quyền ghi, thay đổi (w)
 - 📖 Quyền xóa
 - 📖 Quyền thay đổi chủ file (change owner)

Câu hỏi 2.15 : Trình bày các thao tác cơ bản với file. Phân tích rõ một hệ thống file có nhất thiết phải có thao tác mở file hay không.

📖 Tạo file:

- 📖 Tạo file trống chưa có data; được dành 1 chỗ trong thư mục

📖 Xóa file:

- 📖 Giải phóng không gian mà dữ liệu của file chiếm
- 📖 Giải phóng chỗ của file trong thư mục

📖 Mở file:

- 📖 Thực hiện trước khi ghi và đọc file
- 📖 Đọc các thuộc tính của file vào MEM để tăng tốc độ

📖 Đóng file:

- 📖 Xóa các thông tin về file ra khỏi bảng trong MEM
- 📖 Ghi vào file
- 📖 Đọc file

- **Câu hỏi loại 3 điểm**

Chương 1 :

Câu hỏi 3.1: Trình bày khái niệm hệ điều hành. Phân tích rõ hai chức năng cơ bản của hệ điều hành.

Trả lời :

HDH : phần mềm đóng vai trò trung gian, làm cho việc sd hthong máy tính dc tiện lợi và hiệu quả. Thực hiện 2 chức năng cơ bản

+ Quản lý tài nguyên :

- Đảm bảo cho tài nguyên hệ thống dc sd 1 cách có ích và hiệu quả
- Các tài nguyên :bộ xử lý(CPU),bộ nhớ chính, bộ nhớ ngoài(các đĩa),các thiết bị vào ra.
- Phân phối tài nguyên cho các ứng dụng hiệu quả
 - Yêu cầu tài nguyên dc HDH thu nhận và đáp ứng bằng cách cấp cho chương trình các tài nguyên t/ứ
 - HDH cần lưu trữ tình trạng tài nguyên

+ Quản lý việc thực hiện các chương trình

- 1 c/trình đang trong quá trình chạy gọi là tiến trình
- Hdh giúp việc chạy chương trình dễ dàng hơn
- Tạo ra các máy ảo : là máy logic với các tài nguyên ảo
 - Tài nguyên ảo : mô phỏng tài nguyên dc thực hiện bằng phần mềm
 - Cung cấp các dịch vụ cơ bản như tài nguyên thực
 - Dễ sd hơn
 - Số lượng tài nguyên ảo có thể lớn hơn số lượng tài nguyên thực

Câu hỏi 3.2 : Dịch vụ của hệ điều hành là gì ? Trình bày những dịch vụ điển hình mà hệ điều hành cung cấp. Làm rõ về quá trình tải và chạy hệ điều hành khi mới khởi động.

Trả lời :

+ Các dv điển hình do HDH cung cấp:

- Tải và chạy chương trình

- Để thực hiện trình tự tải từ đĩa vào bộ nhớ, sau đó thực hiện các lệnh.
 - Khi thực hiện xong, cần giải phóng bộ nhớ và các tài nguyên => HDH thực hiện công việc này.
 - HDH tự tải mình vào bộ nhớ
- Giao diện với người dùng
 - Dưới dạng dòng lệnh
 - Giao diện đồ họa
 - Thực hiện các thao tác vào/ra dữ liệu
 - Làm việc với hệ thống file
 - Phát hiện và xử lý lỗi
 - * Phát hiện và xử lý kịp thời lỗi xuất hiện trong phần cứng cũng như phần mềm => Đảm bảo cho hệ thống ổn định, an toàn
 - Truyền thông
 - Cung cấp dịch vụ cho phép thiết lập liên lạc và truyền thông tin
 - Cấp phát tài nguyên
 - Dịch vụ an ninh và bảo mật

Chương 2 :

Câu hỏi 3.3 : Trình bày khái niệm dòng (thread) và mô hình đa dòng. Vấn đề sở hữu tài nguyên của tiến trình và dòng. Phân tích ưu điểm của mô hình đa dòng.

Trả lời :

- + Mỗi đơn vị thực hiện lệnh của tiến trình, tức là 1 chuỗi lệnh được cấp phát CPU để thực hiện độc lập được gọi là 1 luồng thực hiện.
- + Mô hình đa luồng ;
 - Mỗi luồng cần có khả năng quản lý con trỏ lệnh, nội dung thanh ghi
 - Luồng cũng có trạng thái riêng => Chạy trong không gian quản lý luồng.
 - ALL các luồng của 1 tiến trình chia sẻ bộ nhớ và tài nguyên.
 - Các luồng có cùng bộ nhớ địa chỉ và có thể truy cập tới dữ liệu của tiến trình

+ Tài nguyên của tiến trình :

- Trong ht cho phép đa luồng, tiến trình vẫn là 1 đơn vị để HDH phân phối tài nguyên

- Mỗi tiến trình sở hữu chung 1 số tài nguyên :

- Ko gian nhớ của tiến trình(logic); chứa CT, dữ liệu.
- Các tài nguyên khác : các file đang mở, thiết bị I/O

Vẽ hình_19/c2

+ Ưu điểm

- Tăng hiệu năng và tiết kiệm thời gian
- Dễ dàng chia sẻ tài nguyên và thông tin
- Tăng tính đáp ứng
- Tận dụng dc kiến trúc xử lý với nhiều CPU
- Thuận lợi cho việc tổ chức chương trình

Câu hỏi 3.4 : Phân tích các vấn đề cần quan tâm trong sử dụng và quản lý tiến trình đồng thời (concurrent processes) đối với ba dạng tiến trình : tiến trình độc lập có cạnh tranh tài nguyên, tiến trình hợp tác nhờ chia sẻ tài nguyên, và tiến trình hợp tác nhờ trao đổi thông điệp.

Trả lời :

+ Tiến trình cạnh tranh tài nguyên với nhau :

- Đảm bảo loại trừ tương hỗ

- Khi các tiến trình cùng truy cập tài nguyên mà khả năng chia sẻ của tài nguyên đó là có hạn=>phải đảm bảo tiến trình này đang truy cập tài nguyên thì tiến trình khác ko dc truy cập.

- Tài nguyên đó gọi là tài nguyên nguy hiểm. Đoạn chương trình có yêu cầu sd tài nguyên nguy hiểm gọi là đoạn nguy hiểm

- Mỗi thời điểm chỉ có 1 tiến trình nằm trong đoạn nguy hiểm=> loại trừ lẫn nhau.
- Ko xảy ra bế tắc :
 - Bế tắc ; tình trạng 2 hoawcj nhiều tiến trình ko thực hiện tiếp do chờ đợi lẫn nhau.
- ko để đói tài nguyên :
 - Tình trạng chờ đợi quá lâu mà ko đến lượt sd tài nguyên
 - + Tiến trình hợp tác với nhau qua tài nguyên chung
 - Có thể trao đổi thông tin bằng cách chia sẻ vùng nhớ dùng chung(biến toàn thể)
- Đòi hỏi đảm bảo loại trừ tương hỗ
- Xuất hiện tình trạng bế và đói
- Yêu cầu đảm bảo tính chất nhất quán dữ liệu
- Đk chạy đua : Tình huống mà 1 số dòng/tiến trình đọc, ghi dữ liệu sd chung và kết quả phụ thuộc vào thứ tự các thao tác đọc ghi.
=>Đặt thao tác truy cập và cập nhật dữ liệu dùng chung vào đoạn nguy hiểm.
 - + Tiến trình có liên lạc nhờ gửi thông điệp
- Có thể trao đổi thông tin trực tiếp với nhau bằng cách gửi thông điệp
- Ko có y/c loại trừ tương hỗ
- Có thể xuất hiện bế tắc và đói

**Câu hỏi 3.5 : Trình bày giải thuật Peterson cho đoạn nguy hiểm.
Phân tích ưu nhược điểm của phương pháp này.**

Trả lời :

- + Giải thuật :
 - Giải pháp thuộc nhóm phần mềm
 - Đề xuất ban đầu cho đồng bộ 2 tiến trình
 - P0 và P1 thực hiện đồng thời với một tài nguyên chung và một đoạn nguy hiểm chung
 - Mỗi tiến trình thực hiện vô hạn và xen kẽ giữa đoạn nguy hiểm với phần còn lại của tiến trình
 - Y/c 2 tiến trình trao đổi thông tin qua 2 biến chung :

- Turn : xd đến lượt tiến trình nào đc vào đoạn nguy hiểm
- Cờ cho mỗi tiến trình : $flag[i]=true$ nếu tiến trình thứ i y/c đc vào đoạn nguy hiểm.

Code 48/4

+ Ưu điểm :

Thỏa mãn các y/c :

- Đk loại trừ tương hỗ
- Đk tiến triển :
 - P0 chỉ có thể bị P1 ngăn cản vào đoạn nguy hiểm nếu $flag[1]=true$ và $turn = 1$ luôn đúng
 - Có 2 khả năng với P1 ở ngoài đoạn nguy hiểm :
 - + P1 chưa sẵn sàng vào đoạn nguy hiểm $\Rightarrow flag[1]=false$, P0 có thể vào ngay đoạn nguy hiểm
 - + P1 đã đặt $flag[1]=true$ và đang trong vòng lặp while $\Rightarrow turn = 1$ hoặc 0
 - Turn = 0: P0 vào đoạn nguy hiểm ngay
 - Turn = 1: P1 vào đoạn nguy hiểm, sau đó đặt $flag[1]=false \Rightarrow$ quay lại kn 1
- Chờ đợi giới hạn
- + Nhược điểm
- sd trên thực tế tương đối phức tạp
- Đòi hỏi tiến trình đang yêu cầu vào đoạn nguy hiểm phải nằm trong trạng thái chờ đợi tích cực.

- Chờ đợi tích cực; tiến trình vẫn phải sd CPU để ktra xem có thể vào đoạn nguy hiểm?=>lãng phí CPU

Câu hỏi 3.6 : Trình bày phương pháp phát hiện và xử lý bế tắc bằng cách sử dụng đồ thị. Phân tích ưu điểm của phương pháp này so với phương pháp ngăn ngừa bế tắc.

Trả lời :

+ Phát hiện bế tắc :

- TH mỗi dạng tài nguyên chỉ có một tài nguyên duy nhất=> sd đồ thị bd quan hệ chờ đợi lẫn nhau giữa tiến trình
- Xây dựng đồ thị cấp phát tài nguyên :
 - Các nút là tiến trình và tài nguyên
 - Tài nguyên dc nối với tiến trình bằng cung có hướng nếu tài nguyên đc cấp cho tiến trình đó.
 - Tiến trình dc nối với tài nguyên trình bằng cung có hướng nếu tiến trình đang đc cấp cho tài nguyên đó.

+Đồ thị chờ đợi :

- Đc xd từ đồ thị cấp phát tài nguyên = cách bỏ đi các nút t/ứ với tài nguyên và nhập các cung đi đi qua nút bị bỏ
- Cho phép phát hiện tình trạng tiến trình chờ đợi là điều kiện đủ để sinh ra bế tắc
- SD thuật toán phát hiện chu trình trên đồ thì có hướng để phát hiện bế tắc trên đồ thị chờ đợi

Vẽ sơ đồ :

+ Thời điểm phát hiện bế tắc ;

- Bế tắc chỉ có thể xh sau khi 1 tiến trình nào đó yêu cầu tài nguyên và ko đc thỏa mãn.

- => Chạy thuật toán phát hiện bế tắc mỗi khi có y/c cấp phát tài nguyên ko đc tm=> cho phép phát hiện bế tắc ngay khi vừa xảy ra.
 - Chạy thường xuyên làm giảm h/nang ht
 - =>Giảm tần suất chạy thuật toán phát hiện bế tắc :
 - Sau từng chu kì từ vài chục phút tới vài giờ
 - Khi có một số dấu hiệu như hiệu suất sd CPU giảm xuống dưới 1 ngưỡng nào đó
 - + Xử lý khi bị bế tắc :
 - Kết thúc all tiến trình đang bị bế tắc
 - Kt lần lượt từng tiến trình đang bị bế tắc đến khi hết bế tắc
 - HDH phải chạy lại thuật toán phát hiện bế tắc sau khi kt 1 tiến trình
 - HDH có thể chọn thứ tự kt tiến trình dựa trên tiêu trí nào đó.
 - Khôi phục tiến trình về thời điểm trc khi bị bế tắc sau đó cho các tiến trình thực hiện lại từ điểm này :
 - Đòi hỏi HDH lưu trữ trạng thái để có thể thực hiện quay lui và khôi phục về các điểm ktra trc đó.
 - Khi chạy lại, các tiến trình có thể lại rơi vào bế tắc tiếp.
- Lần lượt thu hồi lại tài nguyên từ các tiến trình bế tắc cho tới khi hết bế tắc.

Chương 3 :

Câu hỏi 3.7 : Trình bày kỹ thuật tải trong quá trình thực hiện. Trình bày kỹ thuật liên kết động và thư viện dùng chung. Phân tích rõ ưu điểm mà từng phương pháp đem lại.

Trả lời :

+ Tải trong quá trình thực hiện :

- Hàm chưa bị gọi thì chưa tải vào bộ nhớ
- Chương trình chính dc load vào bộ nhớ và chạy

- Khi có lời gọi hàm :
 - Chương trình sẽ ktra hàm đó dc tải vào chưa
 - Nếu chưa, chương trình sẽ tiến hành tải sau đó ánh xạ địa chỉ hàm vào ko gian chung của chương trình và thay đổi bằng địa chỉ để ghi lại các ánh xạ đó.
 - Lập trình viên đảm nhiệm, HDH cung cấp các hàm thư viện cho tải động
 - + Liên kết động và thư viện dùng chung
 - Liên kết tĩnh : các hàm và thư viện đc lk luôn vò chương trình
 - Lãng phí ko gian cả trên đĩa và MEM trong
 - Trong gd lk, ko kết nối các ham thư viện vào chưa trình mà chỉ chèn các thông tin về hàm thư viện đó.
 - Các modul thư viện đc lk trong quá trình thực hiện:
 - Ko giữ bản sao các modul thư viện mà tiến trình giữ stub chứa thông tin về modul thư viện.
 - Khi stub dc gọi, nó ktra modul t/ứ đã có trong bộ nhớ chưa. Nếu chưa, thì tải phần còn lại về và dùng.
 - Lần tiếp theo cần sd, modul thư viện sẽ đc chạy trực tiếp
 - Mỗi modul thư viện chỉ có 1 bản sao duy nhất chứa trong MEM
 - Cần hỗ trợ từ HDH
- Vẽ hình 8/c3

Câu hỏi 3.8 : Trình bày kỹ thuật phân chương động bộ nhớ. Phân tích ưu nhược điểm của phương pháp này so với phân chương cố định. Khi di chuyển chương sang vị trí khác cần thay đổi thông tin gì trong khối ánh xạ địa chỉ.

Trả Lời :

+ Kỹ thuật :

- Kích thước, số lượng và vị trí chương đều có thể thay đổi

- Khi có y/c, HDH cấp cho tiến trình 1 chương có kích thước đúng bằng tiến trình đó.
- Khi tiến trình kt sẽ tạo vùng trống trong MEM
- Các vùng trống nằm cạnh nhau đc nhập lại thành vùng lớn hơn
- Sd các chiến lược cấp chương
 - Chọn vùng thích hợp đầu tiên
 - Vùng thích hợp nhất
 - Vùng ko thích hợp nhất

Hình vẽ(19/C3)

+ Ưu điểm : Tránh phân mảnh trong

+ Nhược điểm : Gây phân mảnh ngoài : dồn những vùng trống nhỏ thành lớn(nén)

+ Nếu chương bị di chuyển thì nội dung của thanh ghi cơ sở bị thay đổi chứa địa chỉ vị trí mới

Câu hỏi 3.9 : Trình bày kỹ thuật phân chương sử dụng phương pháp kề cận (buddy). Phân tích rõ các điểm giống/khác nhau và ưu nhược điểm của phương pháp này so với phân chương cố định và phân chương động (lưu ý không cần trình bày lại hai phương pháp sau).

Trả lời :

+ Trình bày kỹ thuật :

- Các chương và khối trống có kích thước là lũy thừa của $2^k (L \leq k \leq H)$: 2^L : kích thước nhỏ nhất của chương ; 2^H ; kích thước MEM
- Đầu tiên, toàn bộ ko gian nhớ là 2^H , y/c cấp vùng nhớ S
 - $2^H - 1 < S \leq 2^H$; cấp cả 2^H
 - Chia đôi thành 2 vùng 2^{H-1}
 - Tiếp tục chia đôi tới khi tìm đc vùng thỏa mãn $2^{k+1} < s \leq 2^k$
- Sau 1 thời gian xuất hiện các khối trống có kích thước 2^k
- Tạo ds móc nối các vùng có cùng kích thước
- Nếu có 2 khối trống cùng kích thước và kề nhau thì ghép lại thành 1
- Khi cần cấp, sẽ tìm trong danh sách khối phù hợp nhất; nếu ko tìm khối lớn hơn và cắt đôi.

Câu hỏi 3.10 : Trình bày kỹ thuật phân đoạn bộ nhớ bao gồm cả cấu trúc và cách ánh xạ địa chỉ. Phân tích so sánh ưu nhược điểm của phân đoạn với phân trang.

Trả lời :

+ Kỹ thuật phân đoạn bộ nhớ :

- Cấu trúc :
 - Chương trình thường dc chia thành nhiều phần : dữ liệu, lệnh, ngăn xếp
 - Chia chương trình thành các đoạn theo cấu trúc logic
 - Mỗi đoạn đc phân vào 1 vùng nhớ, có kích thước ko bằng nhau.
 - Mỗi đoạn t/ứ với ko gian địa chỉ riêng, đc phân biệt bởi tên (STT) và độ dài của mình.
 - Các vùng nhớ thuộc các đoạn khác nhau có thể nằm ở vị trí khác nhau.
 - Bộ nhớ đc cấp phát theo từng vùng kích thích thay đổi (giống phân chương động)
 - Chương trình có thể chiếm nhều hơn 1 đoạn và ko cần liên tiếp nhau trong MEM (khác phân chương động)
 - Tránh hiện tượng phân trang mảnh trong
 - Có phân mảnh ngoài

- Dễ sắp xếp bộ nhớ
 - Dễ chia sẻ các đoạn giữa các tiến trình khác nhau
 - Kích thước mỗi đoạn có thể thay đổi mà không ảnh hưởng tới các đoạn khác
- Ánh xạ địa chỉ;
- + Sơ đồ bảng đoạn cho mỗi tiến trình. Mỗi ô tương ứng với 1 đoạn chứa:
- Địa chỉ cơ sở: vị trí bắt đầu của đoạn trong bộ nhớ
 - Địa chỉ giới hạn: độ dài đoạn, sử dụng để chống truy cập trái phép ra ngoài đoạn
- + Địa chỉ logic gồm 2 thành phần(s,o):
- S: số thứ tự/tên đoạn
 - O: độ dịch trong đoạn

Câu hỏi 3.11 : Trình bày kỹ thuật nạp trang theo nhu cầu dùng cho bộ nhớ ảo. Phân tích rõ cùng một lệnh có thể xảy ra nhiều sự kiện lỗi trang không.

Trả lời :

+ Trình bày kỹ thuật :

/*

- Tiến trình được phân trang và chứa trên đĩa
- Khi thực hiện, nạp tiến trình vào MEM : chỉ nạp những trang cần dùng
- Tiến trình gồm các trang trên đĩa và trong MEM : thêm bit P trong khoản mục bảng trang để phân biệt (P=1 : đã nạp vào MEM)
- Quá trình kiểm tra và nạp trang :
 - Tiến trình truy cập tới 1 trang, kiểm tra bit P. Nếu P=1, truy cập diễn ra bình thường. Nếu P=0, xảy ra sự kiện thiếu trang.

Ngắt xử lý thiếu trang :

- HDH tìm 1 khung trống trong MEM
- Đọc trang bị thiếu vào khung trang vừa tìm được
- Sửa lại khoản mục tương ứng trong bảng trang: đổi bit P=1 và số khung đã cấp cho trang.
- Khôi phục lại trạng thái tiến trình và thực hiện tiếp lệnh bị ngắt

*/

Nạp trang hoàn toàn theo nhu cầu :

- Bắt đầu 1 tiến trình mà ko nạp bất kì trang nào vào bộ nhớ
- Khi con trở lệnh đc HDH chuyển tới lệnh đầu tiên của tiến trình để thực hiện, sự kiện thiếu trang sẽ sinh ra và trang t/ư đc nạp vào
- Tiến trình sau đó thực hiện bt cho tới lần thiếu trang tiếp theo

Chương 4 :

Câu hỏi 3.12 : Trình bày cấu trúc thư mục dạng cây và dạng đồ thị không có chu trình. Cấu trúc thư mục dạng không chu trình có ưu điểm gì so với dạng cây ? Thế nào là đường dẫn tuyệt đối và đường dẫn tương đối.

Trả lời :

➤ Thư mục cấu trúc cây:

- Thư mục con có thể chứa các thư mục con khác và các file
- Hthog thư mục đc bd phân cấp như 1 cây: cành là thư mục, lá là file
- Phân biệt khoản mục file và khoản mục của thư mục con: thêm bit db trong khoản mục
 - + 1: khoản mục của thư mục dưới
 - + 0 : khoản mục của file
- Tại mỗi thời điểm, ng dùng làm việc với thư mục hiện thời
- Tổ chức cây trong thư mục cho từng đĩa
 - + Trong hệ thống file như FAT của DÓ, cây thư mục đc xây cho từng đĩa. Hệ thống thu mục đc coi là rừng, mỗi cây trên 1 đĩa.
 - + Linux: toàn ht chỉ gồm 1 cây thư mục

➤ Thư mục cấu trúc ko tuần hoàn(acyclic graph):

- Chia sẻ file và thư mục để có thể xuất hiện ở nhiều thư mục riêng khác nhau
- Mở rộng của cấu trúc cây: là và cành có thể đồng thời thuộc về những cành khác nhau
- Triển khai:
 - + SD liên kết: con trỏ tới thư mục or file khác
 - + tạo bản sao của file và thư mục cần chia sẻ và chứa vào các thư mục khác nhau=>phải đảm bảo tính đồng bộ và nhất quán
- Mềm dẻo nhưng phức tạp
- + Cấu trúc thư mục dạng ko chu trình có ưu điểm là mềm dẻo hơn so với cấu trúc dạng cây.
- + Đường dẫn tuyệt đối :

- Đg dẫn từ gốc của cây thư mục, đi qua các thư mục trung gian, dẫn tới file.
- C:\bc\bin\bc.exe

+ Đg dẫn tương đối:

- Tính từ thư mục hiện thời
- Thêm 2 khoản mục đặc biệt trong thư mục “.”, “..”

Câu hỏi 3.13 : Trình bày phương pháp cấp phát không gian cho file sử dụng danh sách kết nối và sử dụng khối chỉ số (l-node). Hai phương pháp này có điểm gì giống và khác nhau.

So sánh	DS KẾT NỐI	KHỐI CHỈ SỐ
Phương pháp	Khác nhau : - Các khối thuộc 1 file có thể nằm bất kì trên đĩa - Khi file đc cấp thêm khối mới, khối đó đc thêm vào cuối ds - Các khối đc kết nối => ds - Phần đầu mỗi khối chứa con trỏ tới khối tiếp theo - HDH đọc lần lượt từng khối và sd con trỏ để xd khối tiếp theo	- ALL con trỏ tới các khối thuộc về 1 file đc tập trung 1 chỗ - Mỗi file có 1 mảng riêng của mình chứa trong 1 khối gọi là khối chỉ mục(l-node) - Mảng chứa thuộc tính của file và vị trí các khối của file trên đĩa - Ô thứ I của mảng chứa con trỏ tới khối thứ I của file - Chọn kích thước l-node: • Nhỏ: tiết kiệm ko gian nhưng ko đủ con trỏ các khối nếu file lớn • Lớn: với file nhỏ chỉ chiếm 1 vài ô thì lãng phí • Giải pháp: - thay đổi size i-node = sd dslk - Sd i-node có ctruc như sau
	Giống nhau : - Khoản mục của file trong thư mục chứa con trỏ tới khối	

	mục này	
Ưu điểm	- Không bị phân mảnh ngoài - Không yêu cầu biết trước kích thước file lúc tạo - Dễ tìm vị trí cho file, khoản mục đơn giản	- Cho phép truy cập trực tiếp - Các khối thuộc về 1 file không cần nằm liên tiếp nhau
Nhược điểm	- Không hỗ trợ truy cập trực tiếp - Tốc độ truy cập không cao - Giảm độ tin cậy và tính toàn vẹn của hệ thống file	Tốc độ truy cập chậm

Câu hỏi 3.14 : Trình bày về yêu cầu phải đảm bảo tính toàn vẹn của hệ thống file và các phương pháp đảm bảo tính toàn vẹn.

Trả lời :

+ Yêu cầu phải đảm bảo tính bảo toàn :

- Hệ thống file chứa nhiều csdl có mối liên kết $\Rightarrow$ thông tin về liên kết bị hư hại, tính toàn vẹn của hệ thống bị phá vỡ.
- Các khối không có mặt trong danh sách các khối trống, đồng thời cũng không có mặt trong 1 file nào đó.
- 1 khối có thể vừa thuộc về 1 file nào đó vừa có mặt trong ds trống
- HDH có các chương trình kiểm tra tính toàn vẹn của hệ thống file được chạy khi hệ thống khởi động, dựa vào sự cố.

+ Các phương pháp đảm bảo tính toàn vẹn

- Giao tác là 1 tập hợp các thao tác cần phải được thực hiện trọn vẹn cùng nhau.
- Với hệ thống file : mỗi giao tác sẽ bao gồm những thao tác thay đổi liên quan cần thực hiện cùng nhau
- Toàn bộ trạng thái hệ thống file được ghi lại trong file log
- Nếu giao tác không được thực hiện trọn vẹn, HDH sử dụng thông tin từ log để khôi phục hệ thống file về trạng thái không lỗi trước khi thực hiện giao tác

• Câu hỏi loại 4 điểm

Chương 2 :

Câu hỏi 4.1:

a) Trình bày các tiêu chí đánh giá thuật toán điều độ.

- + Lượng tiến trình đc thực hiện xong :
- Số lượng tiến trình thực hiện xong trong 1 đơn vị time
- Đo tính hiệu quả của hệ thống
 - + Hiệu suất sd CPU
 - + Thời gian vòng đời trung bình của tiến trình
- Từ lúc có yêu cầu tạo tiến trình đến khi kết thúc
 - + Thời gian chờ đợi
- Tổng time tiến trình nằm trong trạng thái sẵn sàng và chờ cấp CPU
- Ảnh hưởng trực tiếp của thuật toán điều độ tiến trình
 - + Thời gian đáp ứng
 - + Tính dự đoán đc
- Vòng đời, thời gian chờ đợi, thời gian đáp ứng phải ổn định, ko phụ thuộc vào tải của ht
- + Tính công bằng

Các tiến trình cùng độ ưu tiên phải đc đối xử như nhau

b) Trình bày thuật toán điều độ đến trước phục vụ trước và điều độ có mức ưu tiên.

Đến trc phục vụ trc(FCFS)	Điều độ có mức ưu tiên
- Tiến trình yêu cầu CPU trc sẽ đc cấp trc - HDH sắp xếp các tiến trình sẵn sàng vào hàng đợi FIFO - Tiến trình mới đc sắp xếp vào cuối hàng đợi - Đơn giản đảm bảo tính công bằng - Time chờ tb lớn - =>a/h lớn tới hiệu suất chung của toàn hệ thống	- Mỗi tiến trình có 1 mức ưu tiên - Tiến trình có mức ưu tiên cao hơn sẽ đc cấp CPU trc - Các tiến trình có mức ưu tiên bằng nhau đc điều độ theo FCTS - Mức ưu tiên đc xd theo tiêu trí khác nhau

- Thường là thuật toán điều độ ko phân phối lại

c) Cho các tiến trình với thời gian (độ dài) chu kỳ CPU tiếp theo và số ưu tiên như trong bảng sau (số ưu tiên nhỏ ứng với độ ưu tiên cao). Biết rằng các tiến trình cùng xuất hiện vào thời điểm 0 theo thứ tự P1, P2, P3, P4.

Tiến trình	Thời gian (độ dài)	Số ưu tiên
P1	9	3
P2	1	1
P3	2	2
P4	3	1

Vẽ biểu đồ thể hiện thứ tự và thời gian cấp phát CPU cho các tiến trình khi sử dụng thuật toán : 1) điều độ quay vòng với độ dài lượng tử = 1 ; 2) điều độ theo mức ưu tiên không có phân phối lại. Tính thời gian chờ đợi trung bình cho từng trường hợp.

Câu hỏi 4.2 :

a) Trình bày thuật toán điều độ ưu tiên tiến trình ngắn nhất, thời gian còn lại ngắn nhất.

Điều độ ưu tiên tiến trình ngắn nhất(SPF)	Điều độ ưu tiên thời gian còn lại ngắn nhất
☞ Chọn trong hàng đợi tiến trình có chu kỳ sử dụng CPU tiếp theo ngắn nhất để phân phối CPU ☞ Nếu có nhiều tiến trình với chu kỳ CPU tiếp theo bằng nhau, chọn tiến trình đứng trước ☞ Thời gian chờ đợi trung bình nhỏ hơn nhiều so với FCFS ☞ Khó thực hiện vì phải biết độ dài chu kỳ CPU tiếp: ☞ Trong các hệ thống xử lý theo mẻ:	SFP có thêm cơ chế phân phối lại (SRTF) ☞ Khi 1 tiến trình mới xuất hiện trong hàng đợi, HDH so sánh thời gian còn lại của tiến trình đang chạy với thời gian còn lại của tiến trình mới xuất hiện ☞ Nếu tiến trình mới xuất hiện có thời gian còn lại ngắn hơn, HDH thu hồi CPU của tiến trình đang chạy, phân phối cho tiến trình mới

dựa vào thời gian đăng kí tối đa do lập trình viên cung cấp ☞ Dự đoán độ dài chu kỳ CPU tiếp theo: dựa trên độ dài TB các chu kỳ CPU trước đó ☞ Không có phân phối lại	☞ Thời gian chờ đợi trung bình nhỏ ☞ HDH phải dự đoán độ dài chu kỳ CPU của tiến trình ☞ Việc chuyển đổi tiến trình ít hơn so với RR(quay vòng)
---	---

b) Điều độ theo mức ưu tiên có phân phối lại và không phân phối lại khác nhau thế nào ?

Điều độ theo mức ưu tiên có phân phối lại	Điều độ theo mức ưu tiên ko phân phối lại
- Ưu tiên thời gian còn lại ngắn nhất	- Ưu tiên tiến trình nào có mức ưu tiên cao hơn sẽ đc cấp CPU

c) Cho các tiến trình với độ dài và thời điểm xuất hiện như trong bảng sau

Tiến trình	Thời điểm xuất hiện	Độ dài
P1	0	8
P2	2	4
P3	4	2

Vẽ biểu đồ thể hiện thứ tự và thời gian cấp phát CPU cho các tiến trình khi sử dụng thuật toán : 1) điều độ ưu tiên tiến trình ngắn nhất ; 2) điều độ ưu tiên thời gian còn lại ngắn nhất. Tính thời gian chờ đợi trung bình cho từng trường hợp.

Câu hỏi 4.3:

a) Trình bày về các giải pháp phần cứng (cắm ngắt, sử dụng lệnh máy đặc biệt) cho vấn đề loại trừ tương hỗ và đoạn nguy hiểm.

+ Cắm các ngắt:

☞ Tiến trình đang có CPU: thực hiện cho đến khi tiến trình đó gọi dịch vụ hệ điều hành hoặc bị ngắt

☞ => cấm không để xảy ra ngắt trong thời gian tiến trình

đang ở trong đoạn nguy hiểm để truy cập tài nguyên

☞ Đảm bảo tiến trình được thực hiện trọn vẹn đoạn nguy hiểm và không bị tiến trình khác chen vào

☞ Đơn giản

☞ Giảm tính mềm dẻo của HDH

☞ Không áp dụng với máy tính nhiều CPU

+ Sử dụng lệnh máy đặc biệt (tt)

Logic của lệnh Test_and_Set:

```
Bool Test_and_Set(bool & val)
```

```
{  
    bool temp = val;  
    val = true;  
    return temp;  
}
```

☞ Ưu điểm:

☞ Việc sử dụng tương đối đơn giản và trực quan

☞ Có thể dùng để đồng bộ nhiều tiến trình

☞ Có thể sử dụng cho trường hợp đa xử lý với nhiều CPU nhưng có bộ nhớ chung

☞ Nhược điểm:

☞ Chờ đợi tích cực

☞ Có thể gây đói

b) Sử dụng Test_and_Set để thực hiện loại trừ tương hỗ cho bài toán các triết gia ăn cơm.

```
const int n; //n là số lượng tiến trình
```

```
bool lock;
```

```
void P(int i){ //tiến trình P(i)
```

```
for(;;){ //lặp vô hạn
    while(Test_and_Set(lock)[i]); // lấy đĩa bên trái
    while(Test_and_Set(lock)[(i+1)%5]); // lấy đĩa bên phải
    <Ăn Cơm>
    lock [i]= false;
    lock [(i+1)%5]= false;
    <Suy nghĩ>
}
}
void main(){
for(int i=0;i<5;i++)
    lock [i]= false;
//tắt tiến trình chính, chạy đồng thời n tiến trình
    StartProcess(P(1)); .... StartProcess(P(n));
}
```

c) Phân tích rõ giải pháp sử dụng Test_and_Set sử dụng ở trên có thể gây bế tắc hoặc όχι không.

Sd lệnh test_and_set có thể gây ra tình trạng bế tắc trong TH cả 5 ông cùng lấy 1 đĩa bên phải hoặc bên trái. Như vậy sẽ ko có ông nào đc ăn do vòng while() ko có điều kiện t/m. Ngoài ra lệnh test_and_set gây đói tài nguyên

Câu hỏi 4.4 :

a) Trình bày phương pháp sử dụng cờ hiệu (semaphore) cho vấn đề loại trừ tương hỗ và đoạn nguy hiểm.

Cờ hiệu S là 1 biến nguyên được khởi tạo bằng khả năng phục vụ đồng thời của tài nguyên

Giá trị của S chỉ có thể thay đổi nhờ gọi 2 thao tác là Wait và Signal

Wait(S):

☞ Giảm S đi 1 đơn vị

☞ Nếu giá trị của $S < 0$ thì tiến trình gọi wait(S) sẽ bị phong tỏa

Signal(S):

☞ Tăng S lên 1 đơn vị

☞ Nếu giá trị của $S \leq 0$: 1 trong các tiến trình đang bị phong tỏa được giải phóng

và có thể thực hiện tiếp

Wait và Signal là những thao tác nguyên tử

☞ Để tránh tình trạng chờ đợi tích cực, sử dụng 2 thao tác phong tỏa và đánh thức:

☞ Nếu tiến trình thực hiện thao tác wait, giá trị cờ hiệu âm thì thay vì chờ đợi tích cực nó sẽ bị phong tỏa (bởi thao tác block) và thêm vào hàng đợi của cờ hiệu

☞ Khi có 1 tiến trình thực hiện thao tác signal thì 1 trong các tiến trình bị khóa sẽ được chuyển sang trạng thái sẵn sàng nhờ thao tác đánh thức (wakeup) chứa trong signal

b) Sử dụng cờ hiệu để thực hiện đồng bộ hóa cho bài toán Người sản xuất, người tiêu dùng với bộ đệm hạn chế.

Câu hỏi 4.5 :

a) Trình bày giải pháp sử dụng monitor cho vấn đề loại trừ tương hỗ và đoạn nguy hiểm

- Tiến trình đang thực hiện trong monitor và bị dừng lại để

đợi sự kiện => trả lại monitor để tiến trình khác dùng

Tiến trình chờ đợi sẽ được khôi phục lại từ điểm dừng sau

khi điều kiện đang chờ đợi được thỏa mãn

=> Sử dụng các biến điều kiện

Được khai báo và sử dụng trong monitor với 2 thao tác:

cwait() và csignal()

- x.cwait():

Tiến trình đang ở trong monitor và gọi cwait bị phong tỏa cho tới

khi điều kiện x xảy ra

Tiến trình bị xếp vào hàng đợi của biến điều kiện x

Monitor được giải phóng và một tiến trình khác sẽ được vào

- x.csigal():

Tiến trình gọi csigal để thông báo điều kiện x đã thỏa mãn

Nếu có tiến trình đang bị phong tỏa và nằm trong hàng đợi của x do gọi x.cwait() trước đó sẽ được giải phóng

Nếu không có tiến trình bị phong tỏa thì thao tác csigal sẽ không có tác dụng gì cả

b) Sử dụng monitor để thực hiện loại trừ tương hỗ cho bài toán Người sản xuất, người tiêu dùng với bộ đệm hạn chế.

monitor BoundedBuffer

product buffer[N]; //bộ đệm chứa N sản phẩm kiểu product

int count; //số lượng sản phẩm hiện thời trong bộ đệm

condition notFull, notEmpty; //các biến điều kiện

public:

boundedbuffer() { //khởi tạo

count = 0;

}

void append (product x) {

if (count == N)

notFull.cwait (); //dừng và chờ đến khi buffer có chỗ

<Thêm một sản phẩm vào buffer>

count++;

notEmpty.csigal ();

}

product take () {

if (count == 0)

notEmpty.cwait (); //chờ đến khi buffer không rỗng

<Lấy một sản phẩm x từ buffer>

count --;

```
    notFull.csignal ( );
}
}

void producer ( ) { //tiến trình người sản xuất
    for (;;) {
        <Sản xuất sản phẩm x>
        BoundedBuffer.append (x);
    }
}

void consumer ( ) { //tiến trình người tiêu dùng
    for (;;) {
        product x = BoundedBuffer.take ();
        <Tiêu dùng x>
    }
}

void main() {
    Thực hiện song song producer và consumer.
}
```

Câu hỏi 4.6 :

a) Phòng tránh bế tắc là gì ? Phân tích ưu điểm của việc phòng tránh bế tắc so với ngăn ngừa bế tắc.

Phòng tránh (deadlock avoidance): cho phép một số điều kiện bế tắc được thỏa mãn nhưng đảm bảo để không đạt tới điểm bế tắc

Ngăn ngừa bế tắc	Phòng tránh bế tắc
Sử dụng quy tắc hay ràng buộc khi cấp phát tài nguyên để ngăn	Cho phép 3 điều kiện đầu xảy ra và chỉ đảm bảo sao cho trạng thái

ngừa điều kiện xảy ra bế tắc Sử dụng tài nguyên kém hiệu quả, giảm hiệu năng của tiến trình	bế tắc không bao giờ đạt tới Mỗi yêu cầu cấp tài nguyên của tiến trình sẽ được xem xét và quyết định tùy theo tình hình cụ thể HDH yêu cầu tiến trình cung cấp thông tin về việc sử dụng tài nguyên (số lượng tối đa tài nguyên tiến trình cần sử dụng)
---	--

b) Trình bày phương pháp phòng tránh bế tắc sử dụng thuật toán người cho vay (banker's algorithm), cho ví dụ minh họa cụ thể.

Khi tiến trình muốn khởi tạo, thông báo dạng tài nguyên và số lượng tài nguyên tối đa cho mỗi dạng sẽ yêu cầu
 Nếu số lượng yêu cầu không vượt quá khả năng hệ thống, tiến trình sẽ được khởi tạo

Trạng thái được xác định bởi tình trạng sử dụng tài nguyên hiện thời trong hệ thống:

☞ Số lượng tối đa tài nguyên mà tiến trình yêu cầu:

☞ Dưới dạng ma trận $M[n][m]$: n là số lượng tiến trình, m : số tài nguyên

☞ $M[i][j]$: số lượng tài nguyên tối đa dạng j mà tiến trình i yêu cầu

Số lượng tài nguyên còn lại:

☞ Dưới dạng vector $A[m]$

☞ $A[j]$ là số lượng tài nguyên dạng j còn lại và có thể cấp phát

Lượng tài nguyên đã cấp cho mỗi tiến trình:

☞ Dưới dạng ma trận $D[n][m]$

☞ $D[i][j]$ là lượng tài nguyên dạng j đã cấp cho tiến trình i

Lượng tài nguyên còn cần cấp

☞ Dưới dạng ma trận $C[n][m]$

☞ $C[i][j] = M[i][j] - D[i][j]$ là lượng tài nguyên dạng j mà tiến trình i còn cần cấp

Trạng thái an toàn: trạng thái mà từ đó có ít nhất một

phương án cấp phát sao cho bế tắc không xảy ra

Cách phòng tránh bế tắc:

Khi tiến trình có yêu cầu cấp tài nguyên, hệ thống giả sử tài nguyên được cấp

Cập nhật lại trạng thái & xác định xem trạng thái đó có an toàn?

☞ Nếu an toàn, tài nguyên sẽ được cấp thật

☞ Ngược lại, tiến trình bị phong tỏa & chờ tới khi có thể cấp phát an toàn

VD: SGK trang 83

Chương 3 :

Câu hỏi 4.7 :

a) Trình bày khái niệm phân trang bộ nhớ.(trang 28)

Bộ nhớ vật lý được chia thành các khối nhỏ, kích thước cố định và bằng nhau gọi là khung trang (page frame)

Không gian địa chỉ logic của tiến trình được chia thành những khối gọi là trang (page), có kích thước bằng khung

Tiến trình được cấp các

khung để chứa các trang

của mình.

Các trang có thể chứa trong

các khung nằm rải rác

trong bộ nhớ

HDH quản lý việc cấp phát khung cho mỗi tiến trình bằng bảng trang (bảng phân trang): mỗi ô tương ứng với 1 trang và chứa số khung cấp cho trang đó

Mỗi tiến trình có bảng trang riêng

Duy trì danh sách các khung trống trong MEM

Tương tự như phân chương cố định: khung tương tự chương, kích thước và vị trí không thay đổi

Tuy nhiên kích thước các phần tương đối nhỏ và các phần

cho 1 tiến trình không cần liên tục nhau

Không có phân mảnh ngoài

Có phân mảnh trong

b) Trình bày về ánh xạ địa chỉ khi phân trang bộ nhớ.

Để tính toán địa chỉ hiệu quả, kích thước khung được chọn là lũy thừa của 2

Địa chỉ logic gồm 2 phần:

☞ Số thứ tự trang (p)

☞ Độ dịch (địa chỉ lệch) của địa chỉ so với đầu trang (o)

Nếu kích thước trang là $2n$.

Biểu diễn địa chỉ logic dưới

dạng địa chỉ có độ dài $(m + n)$ bit

☞ m bit cao: biểu diễn số thứ tự trang

☞ n bit thấp: biểu diễn độ dịch trong trang nhớ

Địa chỉ logic số thứ tự trang (p) độ dịch trong trang (o)

Độ dài m n

Quá trình chuyển địa chỉ logic sang địa chỉ vật lý:

☞ Lấy m bit cao của địa chỉ => được số thứ tự trang

☞ Dựa vào bảng trang, tìm được số thứ tự khung vật lý (k)

☞ Địa chỉ vật lý bắt đầu của khung là $k \cdot 2n$

☞ Địa chỉ vật lý của byte được tham chiếu là địa chỉ bắt đầu của khung cộng với địa chỉ lệch (độ dịch)

=> Chỉ cần thêm số khung vào trước dãy bit biểu diễn độ lệch

Kích thước khung là 1KB

Địa chỉ logic được biểu diễn bằng 16 bit

=> Sử dụng 10 bit để biểu diễn địa chỉ lệch ($n=10$)

6 bit biểu diễn STT trang/ khung

Địa chỉ logic

1502

↔ byte 478

trong trang 1

Quá trình biến đổi từ địa chỉ logic sang địa chỉ vật lý được thực hiện bằng phần cứng

Kích thước trang là lũy thừa của 2, nằm trong khoảng từ 512B đến 16MB

Việc tách phần p và o trong địa chỉ logic được thực hiện dễ dàng bằng phép dịch bit

Phân mảnh trong khi phân trang có giá trị trung bình bằng nửa trang

=> giảm kích thước trang cho phép tiết kiệm MEM

Kích thước trang nhỏ => số lượng trang tăng => bảng trang to, khó quản lý

Kích thước trang nhỏ: không tiện cho việc trao đổi với đĩa

Windows 32bit: kích thước trang 4KB

Cơ chế ánh xạ giữa hai loại địa chỉ hoàn toàn trong suốt đối với chương trình

c) **Giả sử không gian nhớ lô gic gồm 4 trang, mỗi trang kích thước 1024B, bộ nhớ vật lý gồm 32 khung. Bảng trang được cho dưới đây :**

0	3
1	0
2	
3	5

Để biểu diễn địa chỉ lô gic trong trường hợp này cần bao nhiêu bit ?

Tính địa chỉ vật lý cho những địa chỉ lô gic sau : 1052, 2500, 4000.

Câu hỏi 4.8 :

a) Trình bày chiến lược đổi trang ít sử dụng trong thời gian cuối.

Đổi trang ít sử dụng nhất trong thời gian cuối (LRU):

☞ Trang bị đổi là trang mà thời gian từ lần truy cập cuối cùng đến thời điểm hiện tại là lâu nhất

☞ Theo nguyên tắc cục bộ về thời gian, đó chính là trang ít có khả năng sử dụng tới nhất trong tương lai

☞ Thực tế LRU cho kết quả tốt gần như phương pháp đổi trang tối ưu
Xác định được trang có lần truy cập cuối diễn ra cách thời điểm hiện tại lâu nhất?

☞ Sử dụng biến đếm:

☞ Mỗi khoản mục của bảng phân trang sẽ có thêm một trường chứa thời gian truy

cập trang lần cuối - Là thời gian logic

☞ CPU cũng được thêm một bộ đếm thời gian logic này

☞ Chỉ số của bộ đếm tăng mỗi khi xảy ra truy cập bộ nhớ

☞ Mỗi khi một trang nhớ được truy cập, chỉ số của bộ đếm sẽ được ghi vào trường

thời gian truy cập trong khoản mục của trang đó

☞ => trường thời gian luôn chứa thời gian truy cập trang lần cuối

☞ => trang bị đổi là trang có giá trị thời gian nhỏ nhất

Sử dụng ngăn xếp:

☞ Ngăn xếp đặc biệt được sử dụng để chứa các số trang

☞ Mỗi khi một trang nhớ được truy cập, số trang sẽ được chuyển lên đỉnh ngăn

xếp

☞ Đỉnh ngăn xếp sẽ chứa trang được truy cập gần đây nhất

☞ Đáy ngăn xếp chính là trang LRU, tức là trang cần trao đổi

☞ Tránh tìm kiếm trong bảng phân trang

☞ Thích hợp thực hiện bằng phần mềm

Vd ; SGK 55

b) Trình bày chiến lược đổi trang sử dụng thuật toán đồng hồ.

Cải tiến FIFO nhằm tránh thay những trang mặc dù đã được nạp vào lâu nhưng vẫn có khả năng sử dụng

- ☞ Mỗi trang được gắn thêm 1 bit sử dụng U

- ☞ Khi trang được truy cập đọc/ ghi: $U = 1$

- ☞ $\Rightarrow$ ngay khi trang được đọc vào bộ nhớ: $U = 1$

- ☞ Các khung có thể bị đổi (các trang tương ứng) được liên kết vào 1 danh sách vòng

- ☞ Khi một trang nào đó bị đổi, con trỏ được dịch chuyển để trỏ vào trang tiếp theo trong danh sách

Khi có nhu cầu đổi trang, HDH kiểm tra trang đang bị trỏ tới

- ☞ Nếu $U=0$: trang sẽ bị đổi ngay

- ☞ Nếu $U=1$: đặt $U=0$ và trỏ sang trang tiếp theo, lặp lại quá trình

Nếu U của tất cả các trang trong danh sách $=1$ thì con trỏ sẽ quay đúng 1 vòng, đặt U của tất cả các trang $=0$ và trang hiện thời đang bị trỏ sẽ bị đổi

Căn cứ vào 2 thông tin để đưa ra quyết định đổi trang:

- ☞ Thời gian trang được tải vào, thể hiện qua vị trí trang trong danh sách giống

như FIFO

- ☞ Gần đây trang có được sử dụng hay không, thể hiện qua bit U

Việc kiểm tra thêm bit U tương tự việc cho trang thêm khả năng được giữ trong bộ nhớ

$\Rightarrow$ thuật toán cơ hội thứ 2

Thuật toán đồng hồ cải tiến:

- ☞ Sử dụng thêm thông tin về việc nội dung trang có bị thay đổi hay không bằng bit M

- ☞ Kết hợp bit U và M, có 4 khả năng:

- ☞ $U=0, M=0$: trang gần đây không được truy cập và nội dung cũng không bị

thay đổi, rất thích hợp để bị đổi ra ngoài

☞ $U=0, M=1$: trang có nội dung thay đổi nhưng gần đây không được truy cập,

cũng là ứng viên để đổi ra ngoài

☞ $U=1, M=0$: trang mới được truy cập gần đây và do vậy theo nguyên lý cục

bộ về thời gian có thể sắp được truy cập tiếp

☞ $U=1, M=1$: trang có nội dung bị thay đổi và mới được truy cập gần đây, chưa thật thích hợp để đổi

Các bước thực hiện đổi trang:

☞ Bước 1:

☞ Bắt đầu từ vị trí hiện tại của con trỏ, kiểm tra các trang

☞ Trang đầu tiên có $U=0$ và $M=0$ sẽ bị đổi

☞ Chỉ kiểm tra mà không thay đổi nội dung bit U , bit M

☞ Bước 2:

☞ Nếu quay hết 1 vòng mà không tìm được trang có U và M bằng 0 thì quét l

danh sách lần 2

☞ Trang đầu tiên có $U=0, M=1$ sẽ bị đổi

☞ Đặt bit U của những trang đã quét đến nhưng được bỏ qua là 0

☞ Nếu chưa tìm được thì lặp lại bước 1 và cả bước 2 nếu cần

c) Giả sử tiến trình được cấp 4 khung nhớ vật lý, các trang của tiến trình được truy cập theo thứ tự sau :

1,2,3,4,5,3,4,1,6,7,8,7,8,9,7,8,9,5. Hãy xác định thứ tự nạp và đổi trang nếu sử dụng hai thuật toán nói trên.

Chương 4 :

Câu hỏi 4.9 (SGK-26)

a) Trình bày phương pháp cấp phát không gian cho file sử dụng các khối liên tiếp. Khi nào nên sử dụng phương pháp này cho hệ thống file ?

Được cấp phát 1 khoảng không gian gồm các khối liên tiếp trên đĩa

Vị trí file trên đĩa được xác định bởi vị trí khối đầu tiên và độ dài (số khối) mà file đó chiếm
Khi có yêu cầu cấp phát, HDH sẽ chọn 1 vùng trống có số lượng khối đủ cấp cho file đó
Bảng cấp phát file chỉ cần 1 khoản mục cho 1 file, chỉ ra khối bắt đầu, và độ dài của file tính = khối
Là cấp phát trước, sử dụng kích thước phần thay đổi
Ưu điểm:

- ☞ Cho phép truy cập trực tiếp và tuần tự
- ☞ Đơn giản, tốc độ cao

Nhược điểm:

- ☞ Phải biết trước kích thước file khi tạo
- ☞ Khó tìm chỗ cho file
- ☞ Gây phân mảnh ngoài:

b) Trình bày các bước cần thiết (trình bày bằng lời, không cần viết mã) để đọc bảng FAT từ thẻ nhớ USB vào bộ nhớ.

c) Giả sử bảng FAT đã được đọc vào bộ nhớ tại địa chỉ « void *fat », viết đoạn chương trình trên C/C++ để liệt kê tất cả các cluster trống trong số N cluster đầu tiên. Giả sử một file bắt đầu tại cluster n, viết đoạn chương trình liệt kê các cluster thuộc về file đó.

Câu hỏi 4.10 :

a) Trình bày phương pháp sử dụng danh sách kết nối trên bảng chỉ số khi cấp phát không gian cho file.

Bảng chỉ số: mỗi ô của bảng ứng với 1 khối của đĩa
Con trỏ tới khối tiếp theo của file được chứa trong ô tương ứng của bảng

Mỗi đĩa logic có 1 bảng chỉ số được lưu ở vị trí xác định
Kích thước mỗi ô trên bảng phụ thuộc vào số lượng khối trên đĩa

Cho phép tiến hành truy cập file trực tiếp: đi theo chuỗi con

trở chứa trong bảng chỉ mục

Bảng FAT (File Allocation Table): được lưu ở đầu mỗi đĩa logic sau sector khởi động

FAT12, FAT16, FAT32: mỗi ô của bảng có kích thước 12, 16, 32 bit

- b) Trình bày các bước cần thiết (trình bày bằng lời, không cần viết mã) để đọc thư mục gốc từ thẻ nhớ USB vào bộ nhớ trong trường hợp hệ thống file của thẻ nhớ là FAT 16.**
- c) Giả sử thư mục gốc của hệ thống file FAT 16 sử dụng tên file độ dài tối đa 8 ký tự đã được đọc vào bộ nhớ tại địa chỉ « void *root ». Viết đoạn chương trình trên C/C++ thực hiện hai việc : in tên và độ dài các file trong thư mục gốc, tìm một file có tên cho trước trong thư mục gốc và số thứ tự cluster đầu tiên của file đó.**

Ghi chú: Ký hiệu (mã) câu hỏi được quy định X.Y

Trong đó : + X tương đương số điểm câu hỏi (X chạy từ 1 đến 5).

+ Y là câu hỏi thứ Y (Y chạy từ 1 trở đi)